

CO3 ASSESSMENT TOOL 1 Experiments-Based Assessment

28. Quick Sort Performance with Adaptive Recursion Limits:

```
main.py
1 import time
2 import sys
3
4 # Increase recursion limit
5 sys.setrecursionlimit(2000)
6
7 max_depth = 0
8
9 def quick_sort(arr, low, high, depth):
10     global max_depth
11
12     if depth > max_depth:
13         max_depth = depth
14
15     if low < high:
16         pivot = partition(arr, low, high)
17         quick_sort(arr, low, pivot - 1, depth + 1)
18         quick_sort(arr, pivot + 1, high, depth + 1)
19
20 def partition(arr, low, high):
21     pivot = arr[high]
22     i = low - 1
23
24     for j in range(low, high):
25         if arr[j] <= pivot:
26             i += 1
27             arr[i], arr[j] = arr[j], arr[i]
28     arr[i], arr[high] = arr[high], arr[i]
29     return i
30
31 if __name__ == '__main__':
32     arr = list(map(int, input().split()))
33     start_time = time.time()
34     quick_sort(arr, 0, len(arr) - 1, 0)
35     end_time = time.time()
36     print("Sorted Array: ", arr)
37     print("Execution Time: ", end_time - start_time)
38     print("Maximum Recursion Depth: ", max_depth)
39
40 ...Program finished with exit code 0
41 Press ENTER to exit console.
```

Analysis:

Quick Sort selects a pivot element and partitions the array into two parts. Each part is sorted recursively. Limiting the recursion depth helps prevent stack overflow and makes the algorithm more reliable for large datasets.

Report Analysis:

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n^2)$

Space Complexity:

$O(\log n)$